



Documentación de Pruebas Unitarias

Centro Adulto Mayor San Juan Pablo II

Proyecto Next.js con Jest y React Testing Library

Generado el 02/09/2026

1. Resumen ejecutivo

Esta documentación describe el estado actual de las pruebas unitarias del proyecto, las áreas cubiertas, la configuración de ejecución, los mocks utilizados y las brechas de cobertura detectadas al momento de generar el reporte.

Suites ejecutadas

12 / 12

Todas las suites pasaron.

Tests ejecutados

127 / 127

No hay tests fallando ni pendientes.

Tiempo total

3.42 s

Ejecución con `--runInBand`.

El estado funcional de la suite es bueno: todas las pruebas automatizadas pasan. El principal punto débil no es estabilidad, sino alcance: la cobertura total sigue siendo baja porque gran parte del frontend y varias rutas API aún no tienen pruebas directas. La funcionalidad de notificaciones, que se agregó de último y había quedado sin cobertura, ya se cerró por completo (pantalla, cron de envío y la utilidad de fechas de la que depende).

2. Stack y configuración

- Framework de pruebas: `jest`
- Preset TypeScript: `ts-jest`
- Entorno DOM: `jsdom`
- Pruebas de UI: `@testing-library/react` y `@testing-library/jest-dom`
- Setup global: `jest.setup.ts`
- Alias de imports: `@/` mapeado a `<rootDir>/`

Configuración	Valor actual	Impacto
Preset	<code>ts-jest</code>	Permite ejecutar tests TypeScript sin build previo.
Coverage provider	<code>v8</code>	Genera el reporte de cobertura usado en este documento.
Test environment	<code>jsdom</code>	Necesario para hooks y componentes React.
Setup file	<code>jest.setup.ts</code>	Centraliza polyfills y mocks globales.
Patrón de pruebas	<code>*.test.ts</code> y <code>*.test.tsx</code>	Detecta pruebas junto al código fuente.

3. Comandos de uso

Objetivo	Comando
Ejecutar toda la suite	<code>npm test</code>
Ejecutar pruebas en orden secuencial	<code>npm test -- --runInBand</code>
Modo watch para desarrollo	<code>npm run test:watch</code>
Generar cobertura	<code>npm run test:coverage -- --runInBand</code>

4. Inventario actual de suites

Archivo de prueba	Tipo	Casos	Enfoque principal
app/(main)/components/ErrorMessage.test.tsx	UI	10	Render, iconos, descripción, botón de acción y estilos.
app/(main)/components/useEscapeKey.test.ts	Hook	6	Escape, activación, limpieza y rerender.
app/(main)/notificaciones/page.test.tsx	UI	12	Control de acceso admin, carga de configuración, validaciones y guardado.
app/api/adultos/route.test.ts	API	8	Autenticación, listado, agrupación de historial y alta de adulto.
app/api/auth/login/route.test.ts	API	9	Login, validaciones, cookie de sesión y último acceso.
app/api/configuracion-notificaciones/route.test.ts	API	9	Lectura de configuración, permisos admin, validación y apagado de notificaciones.
app/api/emails/reporte-medicamentos/route.test.ts	API	10	Token cron, reglas de envío (día, correo destino, sin medicamentos) y envío exitoso.
app/api/medicamentos/route.test.ts	API	12	Lectura, validaciones y creación de prescripciones.
BD/Acceso.test.ts	Datos	17	RPC, consultas, inserciones, updates y deletes sobre Supabase.
lib/auth.test.ts	Utilidad	7	Lectura de sesión y guard de autenticación.
lib/fecha.test.ts	Utilidad	13	Fecha y día de la semana en Costa Rica, momento del día, edad y formato de hora.
lib/userRoles.test.ts	Utilidad	14	Normalización de roles y valores inválidos.

5. Cobertura actual

La cobertura fue regenerada durante la preparación de este documento. El resumen general es el siguiente:

Métrica	Valor	Lectura
Statements	19.33%	Muy baja por gran volumen de pantallas y rutas sin tests.
Branches	83.2%	Alta dentro del código que sí está siendo ejercitado.
Functions	57.53%	Intermedia, concentrada en utilidades y APIs cubiertas.
Lines	19.33%	Baja por el mismo motivo que statements.

Archivos con cobertura completa o casi completa

Archivo	Statements	Branches	Functions	Lines
BD/Acceso.ts	100%	96.55%	100%	100%
app/(main)/components/ErrorMessage.tsx	100%	85.71%	100%	100%

app/(main)/components/useEscapeKey.ts	100%	100%	100%	100%
app/(main)/notificaciones/page.tsx	100%	91.46%	90%	100%
app/api/adultos/route.ts	100%	90.47%	100%	100%
app/api/auth/login/route.ts	100%	100%	100%	100%
app/api/configuracion-notificaciones/route.ts	97.36%	89.65%	100%	97.36%
app/api/emails/reporte-medicamentos/route.ts	100%	87.5%	100%	100%
app/api/medicamentos/route.ts	100%	80.82%	100%	100%
lib/auth.ts	100%	100%	100%	100%
lib/fecha.ts	100%	100%	100%	100%
lib/userRoles.ts	100%	100%	100%	100%

Áreas sin cobertura relevante

- Pantallas principales del frontend: login, home, adultos, historial, medicamentos, forgot-password y reset-password.
- Layouts de aplicación y comportamiento de navegación.
- Componente grande de administración: `UserManagementModal.tsx`.
- Rutas API aún no cubiertas: dashboard, historial, usuarios, logout, session, backup semanal, reporte diario, test-connection y varias rutas con `[id]`.
- Utilidades no cubiertas: `supabase.ts` y `EnumeradoSPs.ts`.

La cifra global de cobertura puede parecer peor de lo que realmente está el núcleo probado, porque `collectCoverageFrom` incluye una porción grande del proyecto que hoy no tiene suite dedicada. El problema no es que los tests actuales sean malos; es que faltan más pruebas sobre interfaces y rutas secundarias.

6. Alcance detallado por suite

6.1 ErrorState.test.tsx

- Verifica render del título.
- Comprueba icono por defecto y personalizado.
- Valida presencia y ausencia de descripción.
- Valida que el botón solo aparezca si existen `actionLabel` y `onAction` .
- Confirma ejecución del callback al hacer click.
- Comprueba estilos base del componente.

6.2 useEscapeKey.test.ts

- Valida que Escape dispare el callback cuando el hook está activo.
- Confirma que otras teclas no disparan la acción.
- Prueba cambios de estado activo via rerender.
- Prueba sustitución de callback en rerender.
- Verifica limpieza del event listener al desmontar.

6.3 lib/auth.test.ts

- Prueba lectura de la cookie de sesión cuando existe y cuando no existe.
- Valida manejo de JSON inválido o malformado.
- Verifica que `requireAuth()` retorne la sesión o lance `No autenticado` .

6.4 lib/userRoles.test.ts

- Prueba normalización de `admin` y `colaborador` .
- Valida mayúsculas, capitalización y espacios.
- Verifica comportamiento con roles inválidos, vacíos, null y undefined.
- Comprueba que `USER_ROLES` solo contenga los dos roles esperados.

6.5 BD/Acceso.test.ts

- Cubre `executeSupabaseQuery()` para éxito, parámetros y error.
- Cubre `getFromTable()` con filtros, orden, límite y error.
- Cubre `insertIntoTable()` para inserción simple, múltiple y error.
- Cubre `updateTable()` con filtro simple, múltiple y error.
- Cubre `deleteFromTable()` con filtro simple, múltiple y error.

6.6 app/api/adultos/route.test.ts

- Valida respuesta 401 cuando no hay sesión.
- Valida lectura de adultos con historial agrupado.
- Valida manejo de error de consulta.
- Valida creación de adulto mayor y persistencia de campos opcionales.

6.7 app/api/auth/login/route.test.ts

- Valida campos requeridos de email y contraseña.
- Valida credenciales incorrectas y usuario inexistente.
- Prueba autenticación correcta con limpieza de `password_hash` en la respuesta.
- Verifica actualización de `ultimo_acceso` .
- Verifica creación de cookie `user_session` con atributos de seguridad.
- Prueba errores de base de datos y excepciones no controladas.

6.8 app/api/configuracion-notificaciones/route.test.ts

- Valida 401 cuando no hay sesión en la lectura de configuración.
- Valida lectura exitosa de la configuración actual.
- Valida 500 cuando falla la consulta a Supabase en la lectura.
- Valida 403 cuando un colaborador intenta modificar la configuración.
- Prueba el flujo de apagar notificaciones guardando `activo: false` .
- Valida rechazo de correo destino inválido.
- Valida rechazo cuando falta el campo `activo` .
- Valida rechazo cuando falta un campo de día de la semana.
- Valida 500 cuando falla la actualización en Supabase.

6.9 app/api/emails/reporte-medicamentos/route.test.ts

- Valida 401 cuando el token del cron es incorrecto.
- Prueba que si las notificaciones están desactivadas, el sistema omite el envío y no llama a `sendMail` .
- Valida que se omita el envío si no existe configuración de notificaciones.
- Valida 500 cuando falla la consulta de la configuración.
- Valida que se omita el envío cuando hoy no corresponde según los días configurados.
- Valida que se omita el envío cuando falta el correo destino.
- Valida 500 cuando falla la consulta de pacientes.
- Valida que se omita el envío cuando ningún paciente tiene medicamentos programados hoy.
- Prueba el envío exitoso del correo con el detalle de medicamentos generado.
- Valida 500 ante un error inesperado no controlado.

6.10 app/api/medicamentos/route.test.ts

- Valida 401 sin sesión.
- Valida lectura conjunta de adultos y prescripciones.
- Valida error cuando falla la consulta de adultos o prescripciones.
- Valida entrada incorrecta para `id_adulto_mayor` .
- Valida nombre requerido del medicamento.
- Prueba normalización de espacios.
- Prueba valores por defecto para horarios no enviados.
- Valida error de inserción y alta exitosa.

6.11 app/(main)/notificaciones/page.test.tsx

- Muestra el estado de carga mientras se verifica la sesión.
- Muestra un estado de acceso restringido cuando el usuario no es admin.
- Carga y muestra el formulario completo cuando el usuario es admin.

- Detecta el modo "días específicos" cuando no todos los días están activos.
- Detecta el modo "todos los días" cuando todos los días están activos.
- Muestra un error con opción de reintentar si falla la carga, y verifica que reintentar recargue.
- Oculta el correo y los días al desactivar el interruptor de notificaciones.
- Verifica que cambiar a "Todos los días" oculte la selección de días específicos.
- Bloquea el guardado y muestra error cuando el correo es inválido.
- Bloquea el guardado cuando está activo en modo específico sin ningún día seleccionado.
- Prueba el guardado exitoso, el payload enviado y el toast de éxito.
- Prueba el toast de error cuando falla el guardado.

6.12 lib/fecha.test.ts

- Valida `obtenerFechaCR()` según la zona horaria de Costa Rica.
- Valida `obtenerDiaSemanaCR()` para los siete días de la semana.
- Valida `obtenerDiaSemanaHoyCR()` con la hora del sistema simulada.
- Valida el formato en español de `formatearFechaLarga()`.
- Valida `obtenerMomentoActualCR()` en distintos horarios, incluida la madrugada sin momento asignado.
- Valida el formato corto de `formatearHoraCR()`.
- Valida `calcularEdad()` antes, después y el mismo día del cumpleaños.

7. Mocks y entorno de prueba

La suite depende de una capa de mocks relativamente fuerte, lo cual permite aislar el comportamiento de la UI y de las rutas sin conectarse a Supabase real ni a APIs de Next.js en tiempo de ejecución.

Mock o polyfill	Ubicación	Propósito
TextEncoder / TextDecoder	jest.setup.ts	Compatibilidad con APIs usadas por Next.js y librerías.
fetch global	jest.setup.ts	Aislar llamadas remotas.
Headers, Request y Response	jest.setup.ts	Simular entorno de rutas API.
next/navigation	jest.setup.ts	Mock del router y hooks de navegación.
next/headers	jest.setup.ts y tests puntuales	Mock de cookies y headers de servidor.
@/lib/supabase	Tests de datos y API	Simular consultas y mutaciones sin tocar la BD.
bcryptjs.compare	Login	Controlar resultado de verificación de contraseña.

8. Riesgos y limites actuales

- Salvo notificaciones, no hay pruebas automatizadas sobre las demás pantallas completas del usuario final.
- No se están cubriendo flujos complejos con estado local, formularios extensos o navegación entre pantallas en el resto de módulos.
- No hay cobertura sobre rutas críticas como usuarios, historial, dashboard, logout o reset de contraseña.
- La cobertura de ramas es buena en el código probado, pero todavía no protege regresiones en la mayor parte de la aplicación.

9. Prioridades recomendadas

Prioridad	Área	Motivo
Alta	app/api/usuarios y app/api/historial	Son rutas de negocio importantes hoy sin cobertura.
Alta	UserManagementModal.tsx	Es un componente grande con permisos, formularios y acciones sensibles.
Alta	Formularios de adultos y medicamentos	Concentran validaciones y mayor probabilidad de regresión visual/funcional.
Media	Dashboard	Importa para cálculos diarios y reportes, pero hoy no está protegido.
Media	Forgot/reset password y logout/session	Ayudan a cubrir autenticación de punta a punta.

10. Evidencia de ejecución usada

Resultado de suite: 12 suites aprobadas, 127 tests aprobados, 0 fallos.

Comando: `npm test -- --runInBand --json --outputFile>manual\test-results.json`

Cobertura: `npm run test:coverage -- --runInBand`

Artefactos: `manual\test-results.json` y carpeta `coverage\`

Documento generado automáticamente a partir de la configuración Jest del proyecto, el inventario real de archivos .test.ts/.test.tsx y la ejecución de la suite en fecha 02/09/2026.